

Arquitetura

1. Arquitetura do Sistema

O sistema foi feito de modo a separar o *frontend* do *backend*. Desta forma a comunicação entre a interface e o servidor é feita recorrendo a uma *API*, mantendo a segurança e os dados sempre protegidos do lado do servidor.

1.1. Estrutura

Frontend → *Angular*: Interface da aplicação. O *Angular* proporciona o *SPA*, que torna a navegação mais rápida e fluída, uma vez que não é preciso carregar a página cada vez que o utilizador muda de ecrã.

No *frontend* também da validação inicial dos formulários e a experiência de quem usa o sistema (alertas).

Backend → *Laravel*: É o motor do projeto. O *Laravel* trata dos *endpoints* necessários para corresponder aos pedidos dos utilizadores, validando o que é preciso e processando toda a informação antes de a devolver à interface.

Base de Dados: É onde guardo as tabelas, recorrendo ao *SQL*. Fiz uso do *Eloquent*, ferramenta própria do *Laravel*, que faz a comunicação com os dados. Desta forma, facilita a gestão das relações existentes.

Também utilizei *seeders*, neste caso para criar o *super admin*.

1.2. Segurança e Controlo de Acesso

Pedidos: Todas as ações que o utilizador faz são enviadas através do protocolo HTTP. Para garantir que o sistema sabe quem está a pedir o quê, faço uso do *Laravel Sanctum*, que envia um *token* em cada pedido.

Validação: Assim que o pedido chega ao servidor, passa pelos *Middlewares*. O *auth:sanctum* verifica se a pessoa está autenticada, e depois uso o *admin.verificar* para garantir que apenas quem é administrador consegue fazer alterações importantes, por exemplo editar ou apagar recursos.

Processamento: Se tudo estiver certo, o *GestaoRecursosController* ou o *GestaoRequisicaoController* tratam do pedido e devolvem a resposta em *JSON*. Se o utilizador não tiver permissão para aquela ação, o sistema devolve logo o erro *403 Forbidden*.

1.3. Validação de Dados

De modo a evitar erros ou dados incorretos/inválidos, faço validações da seguinte forma:

Do lado do utilizador: O *Angular* avisa assim que faltarem dados obrigatórios, evitando sobrecarregar o servidor.

Do lado do servidor: Mesmo que alguém tente contornar a interface, o servidor verifica sempre se os dados fazem sentido antes de os gravar na base de dados. Assim, garanto que o sistema é fiável e que não entram dados errados.